

Development of Microcontroller-Based AI Robot Tour Guide Utilizing Custom Language Models

Ian S. Jackson
West Virginia University
Morgantown, United States
isj0001@mix.wvu.edu

Aiden G. Ballard
West Virginia University
Morgantown, United States
agb00033@mix.wvu.edu

Dr. Mohamed Hefaida
West Virginia University
Morgantown, United States
mohamed.hefeida@mail.wvu.edu

Dr. Anurag Srivastava
West Virginia University
Morgantown, United States
anurag.srivastava@mail.wvu.edu

Dr. Prashnna Gyawali
West Virginia University
Morgantown, United States
prashnna.gyawali@mail.wvu.edu

Abstract—This paper presents the design and implementation of an AI-powered robotic tour guide system for the West Virginia University Lane Department of Computer Science and Electrical Engineering (LCSEE). The system enables prospective students to engage in real-time, natural language conversations with a robot equipped with department-specific knowledge. Three methodologies were explored: (1) a DirectLLM approach that fine-tunes a large language model on a custom dataset, (2) a Classify-Retrieve-Generate (CRG) pipeline that modularizes classification, answer retrieval, and natural response generation, and (3) a Retrieval-Augmented Generation (RAG) approach that retrieves context from plain-text documents scraped from the department’s web pages and conditions a generative model on that context. A custom SQuAD-style dataset was developed using LCSEE data, expanded to 1,477 question-answer pairs across 13 categories, supporting all three pipelines. The system was deployed on a Raspberry Pi 4 integrated with a MangDang Mini Pupper robot. Evaluation results show that while DirectLLM excels in response fluency as measured by BLEU score, it is hindered by retraining requirements and limited scalability. CRG achieves strong retrieval accuracy (99.65%) and fast response times, while RAG achieves a higher BLEU score (19.72) than CRG (16.03) but at the cost of significantly lower retrieval accuracy (44.01%). Future work includes dataset expansion, CRG generator improvements, DirectLLM fine-tuning, and exploration of hybrid fusion models. This research demonstrates the feasibility of deploying compact, domain-aware AI agents for real-time educational engagement.

Index Terms—Robotic Tour Guide Custom Language Models DirectLLM Classify-Retrieve-Generate (CRG) Retrieval-Augmented Generation (RAG) Raspberry Pi Deployment Domain-Specific NLP

I. INTRODUCTION

Advancements in artificial intelligence (AI) and natural language processing (NLP) have enabled new forms of human-computer interaction, particularly in the realm of educational engagement. Universities increasingly seek innovative ways to connect with prospective students, providing them with

immersive experiences that showcase academic programs, research opportunities, and campus life. One such approach is the integration of AI-powered robotic tour guides that allow visitors to interact dynamically with departmental resources.

This research focuses on the development of an AI-driven robotic tour guide designed to enhance the experience of prospective students visiting the West Virginia University Lane Department of Computer Science and Electrical Engineering (LCSEE). The goal is to create an interactive medium where students can ask questions about the department, and the robot, powered by language models, generates informative responses in real-time. Unlike static presentations or scripted responses, this approach enables natural, context-aware interactions, providing a more engaging and personalized tour experience.

To achieve this, three distinct methodologies were explored: (1) DirectLLM, which fine-tunes a large language model (LLM) specifically for department-related queries, (2) a Classify-Retrieve-Generate (CRG) pipeline, which first classifies the user’s question, retrieves relevant information, and then generates a response, and (3) a Retrieval-Augmented Generation (RAG) pipeline, which retrieves relevant context from the dataset using dense vector search and conditions a generative model on that context to produce a response. A custom Q&A dataset was curated using department-specific data, ensuring that the AI model delivers accurate and contextually relevant information.

The system was deployed on a Raspberry Pi 4, paired with a MangDang Mini Pupper [1] robot as the physical embodiment of the tour guide. The robot is equipped with a simple microphone and speaker interface, facilitating natural language communication with users. The choice of hardware necessitated the use of a lightweight AI model optimized for fast inference, ensuring real-time conversational interactions without significant latency.

This paper details the design, development, and deployment of the AI-powered robotic tour guide, evaluating the performance of DirectLLM, the CRG-based approach, and the RAG-based approach. By comparing these methods, we aim to deter-

This research was partially supported by the Defense Advanced Research Projects Agency (DARPA) Award No. 000842. Computational resources were provided by the WVU Research Computing Thorny Flat HPC cluster, partly funded by NSF OAC-1726534.

mine the most effective strategy for delivering responsive and informative AI-driven interactions in an embedded robotics context.

II. RELATED WORK

A. Fine-Tuning Large Language Models for Domain-Specific Tasks

Fine-tuning pre-trained LLMs has become a standard approach to adapting general-purpose models for domain-specific applications. Studies show that fine-tuned LLMs outperform generic models in specialized domains, such as healthcare and legal services [2], by leveraging tailored datasets. The DirectLLM approach in this work applies the same principle, fine-tuning a model specifically on LCSEE information. However, fine-tuning requires significant computational resources, which poses challenges for deployment on edge devices like microcontrollers and single-board computers [3].

B. Deploying AI Models on Resource-Constrained Devices

Deploying AI models on low-power devices such as Raspberry Pi and microcontrollers presents unique challenges, primarily due to hardware limitations in memory and computation. Prior studies have explored optimization techniques such as quantization, pruning, and distillation to reduce model size while preserving accuracy [4]. Recent advancements in small language models (SLMs) offer promising alternatives for efficient, real-time inference [5]. This work integrates such optimizations to deploy a lightweight yet effective AI system on a Raspberry Pi 4.

C. Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) is an approach that augments a generative language model with an external retrieval mechanism, allowing the model to condition its output on retrieved context rather than relying solely on parameters learned during training [16]. This is particularly well-suited for domain-specific applications where factual accuracy is critical and the knowledge base is subject to change. RAG systems typically encode a document corpus into a dense vector index, retrieve the most relevant passages for a given query using similarity search, and pass the retrieved context to a language model for response generation. Prior work has demonstrated that RAG can improve factual grounding compared to purely parametric models [16]. In this work, RAG is evaluated using a corpus of plain-text documents scraped from the department's official web pages, rather than a manually curated Q&A dataset. This design supports automated knowledge base updates via a web scraper, making it a practical option for deployments where the underlying information changes frequently.

III. SYSTEM OVERVIEW

This section presents the overall architecture of the AI-powered robotic tour guide system, detailing the hardware platform, software architecture, and end-to-end data flow from voice input to generated response.

The robot features three main parts to facilitate an interactive experience: a microphone, a computer, and a speaker. A simplified system diagram can be seen in Figure 1. The prospective student will interact with the robot via the microphone, where their question will be transcribed from audio to text. This question is then fed into either the DirectLLM pipeline, CRG pipeline, or RAG pipeline. Once the pipeline processes the prompt, it returns a response to the prospective student via a speaker.

The chosen robotic medium is the MangDang Mini Pupper [1]. The decision was made based on the cost, ease of assembly, and the all-inclusive kit. Speech recognition was handled via the Uberi SpeechRecognition library interfaced with Google's API. For text-to-speech, the Flite engine by CMU was used.

IV. CUSTOM DATASET DEVELOPMENT

Due to the niche nature of the application, a custom dataset of question and answer pairs about the LCSEE department was developed. The dataset was created by curating information from various sources, including the LCSEE department website, course catalogs, departmental brochures, and talking to tour guides of the department. The goal was to ensure that the dataset covered a wide range of topics relevant to prospective students, including academic programs, faculty research, campus facilities, and student life.

Question and answer pairs were human-generated, ensuring accuracy and eliminating the risk of misinformation. To expand the dataset and improve generalization, data augmentation techniques were employed. This included paraphrasing questions, rephrasing answers, and introducing synonyms to create variations of the original pairs. To efficiently augment the dataset, each question answer pair was fed into ChatGPT to generate multiple variations of the same question and answer.

TABLE I: Custom Dataset Summary

Category	Number of Pairs
Degree Programs	153
Research Opportunities	117
Facilities and Resources	157
Clubs and Organizations	205
Career Opportunities	195
Internships	102
Financial Aid and Scholarships	112
Faculty Information	127
Admission Process	150
Location and Contact	49
Generic	81
Follow Up	14
Repeat	15
Total	1,477

The dataset is stored in the SQuAD format [6]. The SQuAD format includes a list of topics as the top level, with each topic having a title. In each topic exist paragraphs, these contain context to the topic and question-answer pairs. To fit the dataset to the task at hand, the topics were chosen based off of types of questions that could be asked about the department: degree programs, research opportunities, facilities

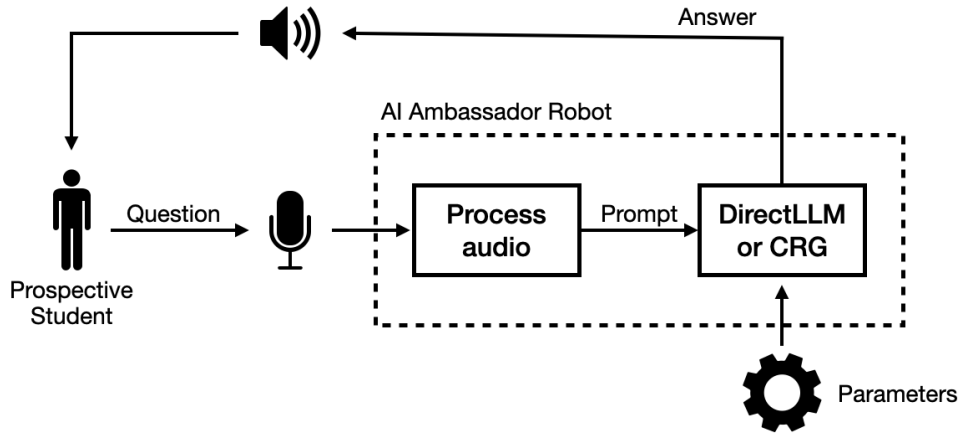


Fig. 1: AI Robot System Diagram

and resources, clubs and organizations, career opportunities, internships, financial aid and scholarships, faculty information, admission process, and location and contact. Two additional categories—follow up and repeat—were introduced to handle conversational continuity and duplicate query patterns observed during testing.

The dataset was expanded significantly from an initial 750 pairs to 1,477 question-answer pairs across 13 categories. A table summarizing the updated dataset can be seen in Table I.

V. DIRECTLLM MODEL

A. Model Selection and Justification

The DirectLLM model consists of a pre-trained LLM that is fine-tuned on the custom dataset. To meet the project’s objectives, the language model was developed based on the following criteria: it needed to be open source, capable of effective training on question-and-answer (Q&A) datasets, and optimized for lightweight deployment with quick response times on a microcontroller platform.

An open-source language model was chosen due to its capabilities in lightweight implementations while maintaining high-quality natural language understanding and generation. Open-source models such as FLAN-T5 [7] and BART [8] provide a solid foundation due to their small size and performance on Q&A task with minimal fine-tuning. These models are well-suited for customization and can be fine-tuned with domain-specific data.

Fine-tuned Language-Agnostic Network (FLAN-T5) is a variant of the T5 (Text-to-Text Transfer Transformer) model, which is pretrained on a diverse range of tasks. It is highly adaptable and can be trained on a wide variety of datasets, including Q&A tasks. It also meets the versatile, scalability, and open-source criteria. The model used in this project has 77 million parameters, making it lightweight and suitable for deployment on resource-constrained devices.

Bidirectional and Auto-regressive Transformer (BART) is a sequence-to-sequence model optimized for tasks like text generation, summarization, and translation. It combines bidirectional context encoding with autoregressive decoding. It

features capabilities to fine-tune on specific datasets, and effective in both understanding and generating coherent responses. Like FLAN-T5, it meets the criteria concerned with being open-source, scalable, and versatile. The model used in this project has 139 million parameters, making it suitable for deployment on moderately resource-constrained devices.

B. Training and Fine-Tuning Process

Both FLAN-T5 and BART were finetuned on the same custom dataset. The model hyperparameters can be found in Table II. The test dataset included 284 questions pertaining to the LCSEE department, with 20 questions from each of the 11 categories.

TABLE II: Hyperparameters for FLAN-T5 and BART Fine-Tuning

Hyperparameter	FLAN-T5	BART
Evaluation Strategy	Epoch	Epoch
Weight Decay	0.01	0.01
Learning Rate	5e-5	3e-5
Batch Size	8	8
Number of Training Epochs	10	5

C. Deployment on Microcontroller

The implementation of FLAN-T5 and BART leverages Hugging Face Transformers, a comprehensive framework for working with pre-trained language models. Training was done via the Hugging Face training pipeline, built on the Trainer class and text generation was configured for inference using the Hugging Face pipeline for text generation. Each model utilized their own tokenizer: The T5 Tokenizer [9] for FLAN-T5 and BART Tokenizer [10] for BART.

VI. CLASSIFY-RETRIEVE-GENERATE (CRG) MODEL

To address the issues of how to handle new information in the LCSEE department, a new approach is needed. Instead of a DirectLLM approach, which would need to be retrained with new information, the classify-retrieve-generate (CRG)

approach is proposed. This approach uses a neural network to *classify* the user’s question into one of the categories from Table I. Once classified, the best-matching answer is then *retrieved* from the custom database based on the question, category, and extracted features. Finally, the retrieved answer is *generated* into a more natural response using a lightweight generative model. This modular design allows classification and retrieval components to be updated independently of the generative model. New information can be added to the database without needing to retrain the entire model. Lastly, the CRG approach allows for a more efficient use of resources, as the generative model can be smaller and more lightweight than a full LLM. The CRG pipeline can be seen in Figure 2.

Each step of the CRG pipeline has various approaches that can be used to achieve the desired results. The following sections detail the best performing approaches for each step of the pipeline. A summary of the CRG pipeline components and methods can be seen in Table IV.

A. Classification Step

To initiate the CRG pipeline, a classifier is used to determine the category of an incoming question. Each category corresponds to a high-level topic (e.g., degree programs, research opportunities) within the LCSEE department’s offerings. Several classification approaches were explored, ranging from traditional machine learning to transformer-based fine-tuned models. In each approach, the input question is first transformed into a numerical format using Term Frequency–Inverse Document Frequency (TF-IDF), word embeddings, or contextual token embeddings.

1) *Traditional Machine Learning Approaches: Logistic Regression for Multi-Class Classification:* Logistic regression is a linear model that predicts the probability distribution over a set of classes using a softmax function. It is particularly well-suited for structured, lower-dimensional feature spaces such as TF-IDF vectors.

Given an input feature vector $\mathbf{x} \in \mathbb{R}^n$, the probability of class $i \in 1, 2, \dots, k$ is given by:

$$P(y_i|\mathbf{x}) = \frac{e^{\beta_i^T \mathbf{x}}}{\sum_{j=1}^k e^{\beta_j^T \mathbf{x}}} \quad (1)$$

where β_i is the weight vector for class i , and k is the number of classes. Training involves minimizing the cross-entropy loss between the predicted and actual distributions.

$$\mathcal{L}_{CE} = - \sum_{i=1}^k y_i \log(P(y = i|\mathbf{x})) \quad (2)$$

TF-IDF is used to convert questions into vector representations before being fed into the classifier. The term frequency $\text{TF}(t)$ and the inverse document frequency $\text{IDF}(t)$ are defined as:

$$\text{TF}(t) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}} \quad (3)$$

$$\text{IDF}(t) = \log \left(\frac{N}{|\{d \in D : t \in d\}|} \right) \quad (4)$$

where $f_{t,d}$ is the frequency of term t in document d , N is the total number of documents, and D is the set of all documents. The TF-IDF vector for a term t in document d is given by:

$$\text{TF-IDF}(t) = \text{TF}(t) \cdot \text{IDF}(t) \quad (5)$$

This method is fast and computationally efficient, making it a strong baseline for classification tasks with small to medium-sized datasets.

Support Vector Machines (SVM): SVMs are another traditional method that finds a hyperplane to separate classes by maximizing the margin. In the multi-class setting, a One-vs-Rest (OvR) strategy is typically employed. The decision function for each class is defined as:

$$f_i(\mathbf{x}) = \sum_{i=1}^k \mathbf{w}_i^T \mathbf{x} + b_i \quad (6)$$

where $\mathbf{w}_i$ is the learned weight vector for class i , and b_i is the bias term. The predicted class is:

$$\hat{y} = \arg \max_i f_i(\mathbf{x}) \quad (7)$$

SVMs use the hinge loss function to enforce a margin between decision boundaries:

$$\mathcal{L}_{hinge} = \sum_{i=1}^k \max(0, 1 - y_i f_i(\mathbf{x})) \quad (8)$$

These models work well when the data is not deeply contextual, and they offer high speed and generalization performance for TF-IDF inputs.

2) *Transformer-Based Classification: BERT (Bidirectional Encoder Representations from Transformers):* BERT is a transformer-based model pretrained on large corpora using masked language modeling [11]. It captures contextual relationships in text by processing input bidirectionally. For classification, the [CLS] token’s embedding is used and passed through a feedforward classifier layer. The model is fine-tuned on the task-specific dataset, with the SQuAD-like format providing labeled classes for supervised training.

Fine-tuning BERT involves updating all transformer weights using a small learning rate and minimizing cross-entropy loss over the predicted class probabilities. While BERT achieves high accuracy, it is computationally heavy and slower during inference, which can be problematic for edge or real-time applications.

DistilBERT: DistilBERT is a lighter and faster version of BERT, trained via knowledge distillation [12]. It retains about 97% of BERT’s performance while being 60% faster and using 40% fewer parameters. Like BERT, the classification token is used to predict the question category, and the model is fine-tuned on the custom dataset.

DistilBERT offers a balance between performance and efficiency, making it suitable for mobile or embedded systems where fast inference is critical.

A comparative summary of the classification methods is presented in Table III. These models were all trained and tested on the same labeled dataset of questions and categories.

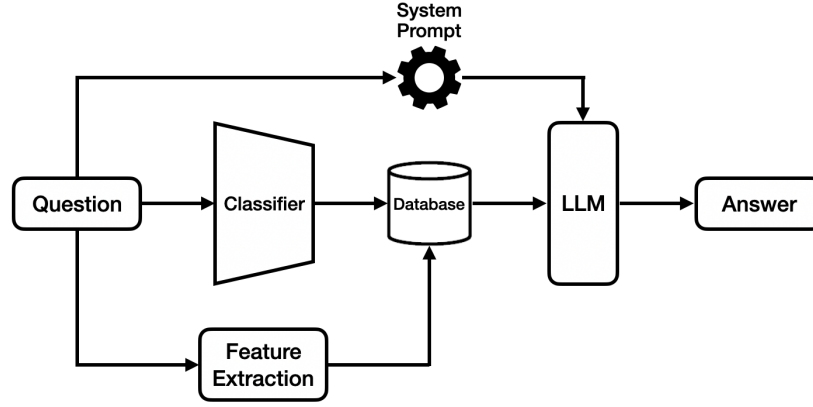


Fig. 2: Classify-Retrieve-Generate (CRG) Pipeline

Evaluation metrics such as accuracy, F1 score, and response time are discussed in Section VIII.

TABLE III: Comparative Summary of Classification Models

Model	Accuracy	Inference Speed
Logistic Regression	Medium	Fast
Support Vector Machine	High	Fast
BERT	Very High	Slow
DistilBERT	High	Medium

B. Feature Extraction Techniques

In the CRG pipeline, once a question is classified into a specific category (e.g., research opportunities, clubs and organizations), the next step is to extract meaningful features from the question to assist in retrieving the most relevant answer. Since each category may contain numerous similar question-answer pairs, surface-level classification alone is insufficient. This section outlines three primary methods explored for extracting relevant information from user queries: Named Entity Recognition (NER), TF-IDF keyword extraction, and word embeddings for semantic search.

1) *Named Entity Recognition (NER)*: Named Entity Recognition (NER) is a technique used to identify and label key terms in a sentence that refer to proper nouns, such as names of people, organizations, degrees, or locations. For this project, spaCy’s pretrained NER model [13] was employed to identify relevant entities within user queries. For example, in the question:

“What research is done in the biometrics field?”

NER extracts: *research, biometrics, field*

“What dual degrees can I pursue?”

NER extracts: *degree*

This method performs well when questions contain distinct, recognizable entities (e.g., Cybersecurity, Morgantown, Capstone), but may struggle with abstract or compound terms (e.g., dual degree, interdisciplinary program). Additionally, the pretrained models are not optimized for the specific academic domain, leading to inconsistent coverage of niche terminology.

2) *TF-IDF Keyword Extraction*: TF-IDF can also be applied to extract high-value keywords from a question. It assigns each word a score indicating its importance in the current question relative to a background corpus.

“What research is done in the biometrics field?”

NER extracts: *research, biometrics, field*

“What dual degrees can I pursue?”

NER extracts: *dual, degree, pursue*

This technique is computationally lightweight and works well with short inputs like student queries. However, its primary limitation is a lack of semantic awareness; it treats each word independently and cannot recognize synonyms or related phrases. For instance, questions about “job prospects” and “career opportunities” may score very differently even if they refer to the same concept.

3) *Word Embeddings for Semantic Search*: To overcome the limitations of sparse keyword methods, semantic embeddings are used to convert entire questions into dense vector representations. The *all-MiniLM-L6-v2* model from Sentence Transformers [14] [15] was chosen for this purpose, as it provides high-quality sentence embeddings that capture semantic meaning effectively. The model generates a 384-dimensional vector for each input question, where semantically similar questions are closer in the vector space.

This method yields the best retrieval results in cases where questions are phrased ambiguously or contain rare synonyms. However, it is computationally more expensive than NER or TF-IDF extraction and requires dimensionality reduction or vector indexing to maintain efficiency in larger datasets

C. Retrieval Methods

Once a question has been classified and its key features extracted, the next step in the CRG pipeline is to retrieve the most relevant answer from the custom question-answer (Q&A) dataset. The retrieval component compares the incoming query—represented via keywords or vector embeddings—against existing questions in the dataset filtered by category. Several techniques were evaluated for this purpose, ranging from simple keyword-based matching to semantic

similarity scoring. This section outlines the retrieval methods implemented and the mathematical formulations behind them.

1) *Exact Keyword Intersection (EKI)*: In the EKI method, the set of extracted keywords from the user query is compared to the keywords in each candidate question (within the same classified category). Each keyword match is given a score of 1, and the total intersection score is used to rank candidates. Let A be the set of keywords from the user query, and B_i be the set of keywords from the i^{th} candidate question. The intersection score is defined as:

$$\text{EKI}(A, B_i) = |A \cap B_i| \quad (9)$$

The question with the highest EKI score is selected. In case of a tie, the first candidate is returned or one is randomly selected. While EKI is fast and easy to implement, it is sensitive to exact word matches and ignores synonyms or paraphrasing.

2) *Jaccard Similarity*: The Jaccard index is a set-based similarity measure used to assess the overlap between two sets of keywords. Unlike EKI, it accounts for both shared and distinct elements. The Jaccard score between sets A and B_i is defined as:

$$\text{Jac}(A, B_i) = \frac{|A \cap B_i|}{|A \cup B_i|} \quad (10)$$

This score ranges from 0 (no overlap) to 1 (complete match). The candidate with the highest Jaccard score is chosen. Ties are handled similarly to EKI. Jaccard provides better granularity than raw intersection and is more robust for uneven keyword set.

3) *JEKI Score (Jaccard + EKI Hybrid)*: To combine the strengths of EKI and Jaccard, a weighted score called the JEKI score was introduced:

$$\text{JEKI}(A, B_i) = \lambda \cdot \text{EKI}(A, B_i) + (1 - \lambda) \cdot \text{Jac}(A, B_i) \quad (11)$$

where $\lambda \in [0, 1]$ is a hyperparameter that balances the two components.

This hybrid approach improves answer selection, especially in tie cases where EKI or Jaccard alone yields similar scores. The balance between raw match count and normalized similarity allows JEKI to adapt across diverse query styles.

4) *Vector Space Retrieval via Cosine Similarity*: For a more semantic retrieval method, user queries and candidate questions can be encoded into dense vector embeddings using pretrained models (as described in Section VI-B3). These vectors are compared using cosine similarity, defined as:

$$\cos(\theta) = \frac{\vec{q} \cdot \vec{q}_i}{\|\vec{q}\| \cdot \|\vec{q}_i\|} \quad (12)$$

where $\vec{q}$ is the vector for the user query and $\vec{q}_i$ is the vector for the i^{th} candidate question.

The question with the highest cosine similarity is retrieved. This method captures contextual closeness even when wording differs significantly (e.g., "student groups" vs. "clubs and organizations"). It requires more memory and computation but yields the highest retrieval accuracy in semantically diverse queries.

D. Generation Step

The final step in the CRG (Classify-Retrieve-Generate) pipeline is the generation stage, which takes the retrieved answer and rephrases it into a more natural, engaging, and conversational response. This layer transforms a static Q&A system into a dynamic tour experience, simulating human-like dialogue. Rather than relying on a large end-to-end model like in the DirectLLM approach, this modular step uses lightweight generation models tailored for refinement, not inference.

The inputs to the generation model include: the original user query, the retrieved answer, and a system prompt that defines the tone and style of the assistant. The generation model is prompted to use the retrieved answer directly, then follow it up with a question or elaboration to encourage ongoing interaction. A sample system prompt used in experimentation was:

"You are an AI assistant for a university tour guide. Your role is to provide engaging and helpful responses based on the retrieved answer. You must always use the retrieved answer in your response first. Then, ask a relevant follow-up question to encourage further conversation."

Several small-scale generative models were tested for compatibility with edge devices. FLAN-T5: Instruction-tuned, lightweight transformer. Easy to deploy but struggles with prompt adherence at smaller sizes. TinyLlama (1.1B): Small, instruction-following model showing early promise in producing coherent responses with minimal compute. DistilBERT: Evaluated but rejected—lacks generative capabilities. GPT-Neo and Mistral-7B: Tested for baseline comparison; too large for final deployment.

All models were run with variations in system prompts and temperature settings to observe response variability and coherence. This step remains work-in-progress. Preliminary results show inconsistent behavior in prompt adherence—many models fail to include the retrieved answer or ask relevant follow-up questions. Further fine-tuning and prompt engineering are planned.

VII. RETRIEVAL-AUGMENTED GENERATION (RAG) MODEL

Retrieval-Augmented Generation (RAG) is introduced as a third approach alongside DirectLLM and CRG. Unlike DirectLLM, which relies on parameters learned during fine-tuning, or CRG, which retrieves from a curated Q&A dataset, the RAG pipeline retrieves context directly from raw plain-text documents scraped from the official LCSEE and Statler College web pages. The retrieved passages are then passed to a generative model to produce a response.

This design choice was motivated by a practical consideration: the department's web pages are already maintained and updated by staff, meaning a web scraper can keep the RAG knowledge base current without any manual curation of Q&A pairs. This is a significant operational advantage over both CRG, which requires human-authored question-answer pairs to

TABLE IV: Summary of CRG Pipeline Components and Methods

Step	Method	Advantages	Limitations	Edge Suitability
Classification	Logistic Regression (TF-IDF)	Fast, good for small datasets	Lower accuracy on complex inputs	High
	SVM (TF-IDF)	Higher accuracy, strong generalization	Ignores context, no semantic understanding	High
	BERT	Very high accuracy, context-aware	Slow inference, large model size	Low
	DistilBERT	Good trade-off of speed and accuracy	Slightly reduced accuracy	Medium
Feature Extraction	NER (spaCy)	Domain-agnostic, fast	Misses abstract academic terms	High
	TF-IDF Keyword Extraction	Lightweight, interpretable	No synonym/semantic handling	High
	Sentence Embeddings	Captures semantics, handles varied phrasing	Higher compute/memory cost	Medium
Retrieval	Exact Keyword Intersection	Simple, fast	Needs exact match, low recall	High
	Jaccard Similarity	Accounts for partial matches	Still keyword-based, no context	High
	JEKI (Hybrid)	Balances raw and normalized	Requires tuning λ	Medium
	Cosine Similarity	Best semantic matching	Most compute-intensive	Medium
Generation	FLAN-T5-Small	Lightweight, open-source	Prompt adherence issues	High
	TinyLlama (1.1B)	Coherent output, promising	Needs fine-tuning	High
	GPT-Neo (1.3B)	Baseline benchmark	Medium resource use, inconsistent	Medium
	Mistral-7B	High-quality responses	Too large for edge deployment	Low

TABLE V: Summary of Response Generation Models (Preliminary)

Model	Params	Output Quality	Edge Suitability
FLAN-T5-Small	80M	Inconsistent	High
TinyLlama	1.1B	Promising	High
GPT-Neo	1.3B	Mixed	Medium
Mistral-7B	7B	High	Low

be added to the database, and DirectLLM, which requires full model retraining whenever departmental information changes.

A. Corpus and Data Source

The RAG corpus consists of plain-text content extracted from eight pages of the LCSEE and Statler College websites, covering the following topic areas: general department information, undergraduate programs, graduate programs, research, faculty and staff, student life, alumni and friends, and department contact information. The text was collected using a web scraper and stored as flat text files, one per page. No human annotation or question-answer pair formatting was applied. This unstructured format is intentional: the goal is to evaluate whether a generative model can produce useful responses directly from raw web content, as would be the case in a fully automated deployment where a scraper periodically refreshes the knowledge base.

B. Architecture Overview

The RAG pipeline consists of two primary components: a retriever and a generator. The retriever chunks the scraped plain-text documents into passages and encodes them into a dense vector index using the *all-MiniLM-L6-v2* sentence embedding model [14] [15], the same model used in the CRG feature extraction step (Section VI-B3). At inference time, the user query is embedded into the same vector space, and the top- k most similar passages are retrieved using cosine similarity. The retrieved passages are concatenated to form a context block, which is then provided to a generative language

model alongside the original query and a system prompt instructing the model to ground its response in the provided context.

The generator used in the RAG pipeline belongs to the same class of lightweight models explored in the CRG generation step, selected for compatibility with edge deployment constraints.

C. Differences from CRG

The RAG and CRG pipelines differ in two fundamental ways: the format of the knowledge base and the retrieval strategy.

CRG retrieves from a structured set of 1,477 human-authored question-answer pairs, each assigned to one of 13 categories. Retrieval is first scoped by the classifier to a single category, then ranked within that category. RAG, by contrast, retrieves from unstructured plain-text passages with no category labels and no classification step. Retrieval is performed globally across all passages using embedding similarity alone.

The consequence of this difference is that CRG’s retrieval is highly precise within a well-classified category, but depends on the classifier making the correct assignment. RAG’s retrieval is more flexible and operates over a broader and more naturally expressed knowledge base, but must identify the most relevant passage from a larger and less uniform pool of candidates. Additionally, because the RAG corpus consists of prose web content rather than curated Q&A pairs, the retrieved context is less likely to contain a direct answer in the same form as the reference answer used for BLEU evaluation, which affects the interpretation of RAG’s BLEU score relative to CRG and DirectLLM.

VIII. EVALUATION AND COMPARISON

This section compares the performance of the DirectLLM, CRG, and RAG approaches in addressing questions about the LCSEE department. The evaluation framework considers both quantitative metrics (accuracy, BLEU, response time, and

memory usage) and qualitative observations (response fluency, robustness, ease of update). The DirectLLM, CRG and, RAG pipelines were each evaluated over a set of 284 questions drawn from the expanded dataset.

A. DirectLLM Evaluation

To evaluate the performance of FLAN-T5 and BART, two metrics are employed: Answer accuracy and BLEU Score. These metrics provide insight to the models’ understanding and generation tasks. Another metric used to evaluate the model is average response time, or the mean of the time it takes to process an input and provide an output over all test data. Lastly, the average memory usage is recorded to determine the feasibility of deploying the model on a microcontroller.

BLEU Score: The Bilingual Evaluation Understudy (BLEU) Score measures the quality of the text generated by comparing it with one or more of the reference texts. It assesses how similar the generated text is to human-provided references, in this case the reference is the provided answer to the test question. The BLEU Score formula is seen in Equation 13.

$$\text{BLEU} = \text{BP} \cdot \exp \left(\sum_{n=1}^N w_n \log(p_n) \right) \quad (13)$$

$$\text{BP} = \begin{cases} 1, & c > r \\ e^{1-r/c}, & c \leq r \end{cases} \quad (14)$$

where BP is the Brevity Penalty (seen in Equation 14), p_n is the precision of n -grams, and w_n is the weight associated to n -grams (typically equal $\forall n$).

B. CRG Evaluation

The various combinations of implementations for each step in the CRG pipeline were evaluated to determine their effectiveness in answering questions about the LCSEE department. In total there are 96 combinations of the CRG pipeline, the best three combinations are shown in this section. Table VI summarizes the three best-performing CRG configurations, including the classification model, extraction method, and retrieval method used in each case.

TABLE VI: Best Performing CRG Combinations

CRG Model	Classification	Extraction	Retrieval
CRG-1	LR	Vector	Cosine
CRG-2	SVM	NER	EKI
CRG-3	DistilBERT	Vector	Cosine

The retrieval is evaluated using a retrieval score (RS), which is a binary score indicating whether the correct answer was retrieved from the dataset.

$$\text{RS} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(y_i = \hat{y}_i) \quad (15)$$

Response quality is evaluated using the BLEU score. All three CRG configurations, DirectLLM, and RAG are also evaluated by their mean end-to-end response time and average memory

usage over the 284-question test set. The DirectLLM results are seen in Table VII, the CRG results in Table VIII, and the RAG results in Table IX. A unified comparison of BLEU scores across all three approaches is provided in Table X.

TABLE VII: DirectLLM Evaluation

Model	Acc.	BLEU	Resp. Time (s)	Mem (MB)
FLAN-T5	68.18%	91.17%	38.45	2115.1
BART	27.64%	74.41%	9.961	648.86

TABLE VIII: CRG Evaluation

Model	RS	BLEU	Resp. Time (s)	Mem (MB)
CRG-1	0.9965	18.96	0.312	755.52
CRG-2	0.9965	10.24	0.578	899.38
CRG-3	0.6092	16.73	0.300	930.56

C. RAG Evaluation

The RAG pipeline was evaluated on the same 284-question test set used for CRG evaluation. Because the RAG corpus consists of unstructured plain-text web pages rather than curated Q&A pairs, the retrieval score (RS) for RAG measures whether the retrieved passage contained information sufficient to answer the question correctly, as judged against the reference answers from the Q&A dataset. Response quality is assessed using the BLEU score.

TABLE IX: RAG Evaluation

Model	RS	BLEU	Resp. Time (s)	Mem (MB)
RAG	0.4401	19.72	0.383	775.22

TABLE X: BLEU Score Comparison Across All Approaches

Approach	Model	BLEU Score
DirectLLM	FLAN-T5	91.17%
	BART	74.41%
CRG	CRG-1	18.96
	CRG-2	10.24
	CRG-3	16.73
RAG	—	19.72

IX. DISCUSSION

A. Lessons Learned

Through the development and evaluation of the DirectLLM, Classify-Retrieve-Generate (CRG), and Retrieval-Augmented Generation (RAG) approaches, several key insights emerged regarding their suitability for deployment in a robotic tour guide system.

One of the primary challenges with the DirectLLM pipeline was the training bottleneck imposed by dataset requirements. As a fine-tuned small language model (SLM), DirectLLM

demands a substantial and diverse dataset to maintain conversational accuracy across a wide range of questions. Furthermore, any updates to department-specific information (e.g., new faculty, facilities, or programs) would necessitate retraining the entire model. This approach is neither scalable nor maintainable for dynamic environments, particularly when operating on constrained hardware.

In contrast, the CRG framework introduced a modular alternative. By decoupling classification, retrieval, and generation into distinct steps, CRG enables individual components to be updated or replaced without the need to retrain an end-to-end model. For example, new answers can be added directly to the retrieval database without affecting the classifier or generator. This modularity makes CRG especially well-suited for educational deployments where content frequently evolves. CRG-1 and CRG-2 both achieved a retrieval accuracy of 99.65% on the 284-question test set, demonstrating that the combination of a classifier and semantic retrieval is highly effective at surfacing the correct answer. CRG-3, which uses DistilBERT for classification, achieved a lower retrieval accuracy of 60.92%, suggesting that the heavier transformer-based classifier does not provide a benefit in retrieval accuracy on this dataset relative to the lighter approaches.

The RAG pipeline offered a different perspective on this trade-off. By removing the classification step and retrieving from raw scraped web pages rather than curated Q&A pairs, RAG represents the most maintainable of the three approaches: the knowledge base can be refreshed automatically by re-running a web scraper whenever the department’s website is updated, with no human annotation required. However, RAG’s retrieval accuracy was substantially lower at 44.01%, compared to 99.65% for the best-performing CRG configurations. This gap is expected given the nature of the two corpora: CRG retrieves from 1,477 carefully authored question-answer pairs that are semantically close to the test questions, while RAG retrieves from unstructured prose passages that may express the same information in a very different form. The retrieval step in RAG must therefore identify a relevant passage from a larger and less uniform pool of candidates, without any category narrowing to guide the search. RAG’s BLEU score of 19.72 was marginally higher than the best-performing CRG configuration (CRG-1: 18.96), though the scores are not strictly comparable given the different corpora used for generation.

B. Trade-Offs

While CRG offers clear advantages in adaptability and maintainability, it comes with certain trade-offs. DirectLLM, once trained, can provide highly fluent and contextually rich responses, particularly for open-ended questions. The model leverages deep contextual embeddings to infer nuanced intent and generate more natural responses. This is reflected in the BLEU scores: FLAN-T5 achieved 91.17% and BART achieved 74.41%, both significantly higher than any CRG configuration (CRG-1: 18.96, CRG-2: 10.24, CRG-3: 16.73) and RAG (19.72).

CRG, though more structured and sometimes less fluent in response style, allows for deterministic output based on a well-controlled retrieval mechanism. Among the CRG configurations, CRG-1 (LR + Vector + Cosine) achieved the highest BLEU score (18.96) while maintaining 99.65% retrieval accuracy, making it the strongest overall performer. CRG-2 (SVM + NER + EKI) matched CRG-1’s retrieval accuracy but produced a notably lower BLEU score (10.24), suggesting that the NER-based extraction and EKI retrieval combination, while fast, produces context that is less useful for the generator. Additionally, CRG’s modular design permits fine-grained improvements to individual components such as classification accuracy or retrieval algorithms without affecting the entire pipeline.

RAG occupies a distinct position in this trade-off space. It is the simplest to maintain of the three approaches, as its knowledge base is populated directly from web-scraped plain-text and requires no manual Q&A authoring or model retraining. Its BLEU score (19.72) was marginally higher than even the best CRG configuration (CRG-1: 18.96), though as noted above these scores are not directly comparable given the different generation corpora. However, RAG’s retrieval accuracy of 44.01% was substantially lower than CRG-1 and CRG-2, which is a meaningful concern in an educational context where providing an incorrect or off-topic answer to a prospective student could undermine confidence in the system.

C. Practical Considerations

For deployment on microcontroller-based robots, several practical considerations must be addressed. Inference time and memory usage are critical constraints. DirectLLM models, such as BART and FLAN-T5, exhibited slower average response times (9.96–38.45 seconds) and higher memory footprints, making them poorly suited for real-time interaction on the Raspberry Pi 4. In contrast, all three CRG configurations completed responses in under 0.6 seconds on average, with CRG-3 being the fastest at 0.300 seconds and CRG-1 close behind at 0.312 seconds. CRG-1 also offers the best balance of speed, memory usage (755.52 MB), retrieval accuracy (99.65%), and BLEU score (18.96), making it the recommended configuration for deployment. CRG-2, while matching CRG-1’s retrieval accuracy, required more time (0.578 seconds) due to the NER extraction step and consumed more memory (899.38 MB), with a significantly lower BLEU score (10.24).

The RAG pipeline achieved an end-to-end response time of 0.383 seconds on average, with a memory footprint of 775.22 MB, placing it between CRG-1 and CRG-2 in resource usage. The lower retrieval accuracy (44.01%) is in part a consequence of the corpus design: because RAG retrieves from unstructured web prose, the relevant information may be spread across a passage in a way that does not align closely with the embedding of a short, direct question. This suggests that retrieval quality could be improved through better chunking strategies or passage preprocessing, without changing the fundamental scraper-based update workflow.

Furthermore, CRG’s deterministic logic and rule-based fallback mechanisms enhance its reliability, which is crucial in educational contexts where incorrect or hallucinated responses could erode user trust. RAG, while operationally simpler to maintain, is more susceptible to retrieval failures on an unstructured corpus, and its generated responses showed a shorter average length (35.42 characters) compared to CRG-1 (54.68 characters) and CRG-2 (51.15 characters), suggesting that the generator may be producing less complete answers when conditioned on prose passages.

X. CONCLUSION

This research presented the development of an AI-powered robotic tour guide for the Lane Department of Computer Science and Electrical Engineering, leveraging three natural language processing (NLP) pipelines: DirectLLM, Classify-Retrieve-Generate (CRG), and Retrieval-Augmented Generation (RAG). Each approach was designed, implemented, and evaluated with the goal of enabling natural, accurate, and efficient interactions between prospective students and the department’s offerings. The evaluation was conducted on an expanded dataset of 1,477 question-answer pairs across 13 categories.

The DirectLLM approach demonstrated strong performance in generating fluent and context-rich responses, with FLAN-T5 achieving a BLEU score of 91.17% and BART achieving 74.41% on the 22-question test set. However, this pipeline faces a critical training bottleneck: any update to the departmental information requires full model retraining, making it inefficient and unsuitable for dynamic content environments. Additionally, DirectLLM response times of up to 38.45 seconds on the Raspberry Pi 4 present a significant barrier to real-time deployment.

The CRG pipeline offers a modular architecture that separates classification, retrieval, and generation into discrete components. This separation enables more efficient updates—new content can be added directly to the retrieval database without retraining. CRG-1 and CRG-2 both achieved a retrieval accuracy of 99.65% across 284 test questions, with response times well under one second for all three configurations. CRG-1 (LR + Vector + Cosine) emerged as the strongest overall configuration, achieving the highest BLEU score among the CRG models (18.96) at a response time of 0.312 seconds and a memory footprint of 755.52 MB, making it well-suited for real-time deployment on the embedded hardware used in this system.

The RAG pipeline offered the most maintainable architecture of the three approaches, retrieving context from plain-text documents scraped directly from the LCSEE and Statler College web pages. This design allows the knowledge base to be refreshed automatically without any manual Q&A curation or model retraining. RAG achieved a BLEU score of 19.72 and a response time of 0.383 seconds. However, its retrieval accuracy of 44.01% was substantially lower than CRG’s, reflecting the difficulty of identifying relevant passages from

unstructured web prose without the guidance of category labels or curated answer formatting.

Future Work

To enhance the robustness and usability of the tour guide system, several directions are proposed for future research: Expand the custom dataset further with more diverse and detailed question-answer pairs to improve both DirectLLM and CRG coverage. Finish development of the CRG generation step, ensuring it reliably incorporates retrieved answers and poses engaging follow-up questions. Further fine-tune the DirectLLM models, particularly BART and FLAN-T5, using the expanded dataset to improve their fluency and domain-specific relevance. Improve classification accuracy in the CRG pipeline, which involves further fine-tuning. Improve the RAG retrieval pipeline by experimenting with passage chunking strategies, passage preprocessing, and hybrid retrieval methods that combine dense embeddings with sparse keyword matching (e.g., BM25), with the goal of improving retrieval accuracy on the unstructured web corpus. Develop an automated scraper pipeline that periodically updates the RAG corpus from the LCSEE website, and evaluate whether retrieval accuracy improves as the corpus coverage grows. Optimize code efficiency in each pipeline stage, especially in retrieval and generation, to meet real-time requirements on embedded hardware. Explore hybrid fusion models that combine the strengths of all three pipelines—such as using CRG’s classifier to scope retrieval for structured queries while falling back to RAG over the web corpus for questions outside the curated dataset’s coverage—to balance performance and flexibility. Together, these enhancements aim to advance the system toward a fully autonomous, scalable, and engaging AI tour guide platform deployable across academic institutions.

REFERENCES

- [1] Mangdang, "Mangdang Store," Available: <https://mangdang.store>
- [2] M. Rajkomar et al., "Scalable and accurate deep learning with electronic health records," *npj Digital Medicine*, vol. 1, no. 18, 2018.
- [3] T. Dettmers et al., "Sparse fine-tuning for efficient deployment of large-scale language models," *NeurIPS Conference Proceedings*, 2022.
- [4] Z. Jiang et al., "Efficient deep learning inference on edge devices: Challenges and techniques," *ACM Computing Surveys*, vol. 55, no. 6, 2023.
- [5] H. Touvron et al., "Llama 2: Open foundation and fine-tuned chat models," *arXiv:2307.09288*, 2023.
- [6] R. Rajpurkar, "SQuAD: The Stanford Question Answering Dataset," *SQuAD Explorer*.
- [7] Chung, Hyung Won, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Eric Li, et al. "Scaling Instruction-Finetuned Language Models." *arXiv* (2022). <https://doi.org/10.48550/arXiv.2210.11416>.
- [8] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, "BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension," *CoRR*, vol. abs/1910.13461, 2019. [Online]. Available: <http://arxiv.org/abs/1910.13461>.
- [9] H. Xu, M. Scales, A. Nguyen, R. Ritter, C. Raffel, K. Shankar, S. A. Saurous, and Q. V. Le, "Deep Evolutionary Reinforcement Learning," *arXiv preprint arXiv:1910.10683*, 2019. [Online]. Available: <https://arxiv.org/pdf/1910.10683>.

- [10] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, "BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension," arXiv preprint arXiv:1910.13461, 2019. [Online]. Available: <https://arxiv.org/pdf/1910.13461>. [Accessed: Dec. 17, 2024].
- [11] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," arXiv preprint arXiv:1810.04805, 2019. [Online]. Available: <https://arxiv.org/abs/1810.04805>
- [12] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," arXiv preprint arXiv:1910.01108, 2020. [Online]. Available: <https://arxiv.org/abs/1910.01108>
- [13] M. Honnibal and I. Montani, "spaCy 2: Natural language understanding with Bloom embeddings, convolutional neural networks and incremental parsing," 2017.
- [14] N. Reimers and I. Gurevych, "Making Monolingual Sentence Embeddings Multilingual using Knowledge Distillation," in Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2020. [Online]. Available: <https://arxiv.org/abs/2004.09813>
- [15] Hugging Face, "all-MiniLM-L6-v2," [Online]. Available: <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>. [Accessed: 01-Apr-2025].
- [16] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.